Idea2PRD — Project Report

Course Name: IIIT LLM Course **Project**: Idea2PRD – Multi-Agent Product Requirement Generator

1. Problem Statement

Product Requirements Documents (PRDs) are essential artifacts in software development, serving as the foundational blueprint for engineering, design, and business teams. However, writing a comprehensive PRD is a time-consuming, labor-intensive process that requires synthesizing abstract ideas, market research, business rules, and technical constraints into a structured, actionable format.

Product Managers often struggle with the "blank page" problem, missing edge cases, or failing to align business goals with technical realities. The process is further complicated when incorporating external knowledge, such as competitor analysis or shifting business guidelines.

The Solution: Idea2PRD addresses this problem by automating the PRD creation process using a multi-agent Large Language Model (LLM) architecture. By breaking down the task into specialized agent roles (Business Analyst, Product Manager, and Risk Reviewer) and grounding their knowledge using Retrieval-Augmented Generation (RAG), Idea2PRD transforms a brief product idea into a structured, professional-grade PRD. It accelerates the ideation phase, ensures comprehensive coverage of requirements, and maintains context through external document injection.

2. Architecture

Idea2PRD is built using a modern, scalable architecture designed around sequential agent orchestration and RAG.

2.1 High-Level Data Flow

The system takes user inputs (product idea, target users, domain, and constraints) alongside optional supporting documents (PDF/TXT). The frontend orchestrates a pipeline where documents are processed into a vector store, and a chain of three LLM agents sequentially processes the idea, passing outputs forward to build the final document.

2.2 Core Components

1. **Frontend Layer (Streamlit)**: Provides a clean, interactive user interface. It handles user inputs, file uploads, API key management, and displays the generated PRD across multiple tabs, including the raw agent outputs and retrieved contexts. It supports multiple LLM providers (Google Gemini, OpenAI, and Groq).

2. **RAG Pipeline (`rag_utils.py`)**:

    - **Document Parsing**: Extracts raw text from uploaded PDFs (via PyPDF2) and TXT files.
    - **Text Chunking**: Utilizes LangChain's `RecursiveCharacterTextSplitter` with a chunk size of 1000 characters and 200 characters overlap to preserve semantic boundaries.
    - **Vector Database**: Chunks are embedded using `models/embedding-001` (Gemini) or `text-embedding-3-small` (OpenAI) and stored in an in-memory **FAISS** vector store. (Note: When using Groq for inference, the system falls back to Gemini's free embeddings).
    - **Retrieval**: For each agent step, the system performs a similarity search to retrieve the top 5 most relevant chunks to inject into the prompt.

3. **Multi-Agent Orchestration (`agents.py`)**: A sequential pipeline

of specialized agents:

- **Business Analyst Agent**: Responsible for defining the problem statement, business goals, target personas, and core assumptions.
- **Product Manager Agent**: Takes the BA's output and generates structured user stories, functional and non-functional requirements, and defines the Minimum Viable Product (MVP) scope.
- **Risk Reviewer Agent**: Reviews both previous outputs to identify potential technical/business risks, ambiguities, missing requirements, and proposes mitigation strategies.
- **Compiler**: A deterministic step that merges the three outputs into a cohesive Markdown document.

---

3. Prompt Design

The system relies heavily on structured prompt engineering to ensure the LLMs produce consistent, high-quality outputs. The prompts are located in `prompts.py` and utilize several advanced techniques:

3.1 Persona-Based Prompting (Role Assignment)

Each agent is given a strict persona. For example, the Product Manager agent is told: *"You are an expert Product Manager. Your job is to take the Business Analyst's output and translate it into actionable product requirements..."* This focuses the model's behavior and tone.

3.2 Context Grounding & Hallucination Guardrails

To prevent the model from inventing facts when RAG context is provided, prompts include a dynamic "Context Injection Block":

- **If context exists**: *"You MUST use this context to ground your analysis. When you use information from these excerpts, explicitly*

cite the source..."
- **If no context exists**: *"You must rely on your general knowledge. When doing so, clearly state your assumptions..."*

3.3 Structured Output Formatting (Few-Shot Guidance)

The prompts dictate the exact Markdown headers and structure the model must output. For instance, the Risk Reviewer is instructed to output strictly:

1. `### Identified Risks`
2. `### Ambiguities & Missing Requirements`
3. `### Open Clarifying Questions`

3.4 Negative Constraints

Prompts include explicit negative instructions to prevent unwanted behavior. For example: *"Do NOT generate requirements or user stories yet"* (for the BA agent), or *"Do NOT invent new features that contradict the core idea."*

---

4. Evaluation

Evaluating generative AI pipelines requires a combination of automated checks and qualitative human review. Idea2PRD implements a hybrid evaluation framework:

4.1 Internal Quality Checks

The system includes a `QUALITY_CHECKLIST` (defined in `config.py`) that acts as a heuristic evaluation tool. The final PRD is reviewed against criteria such as:

- Are there clear user personas defined?

- Are user stories formatted correctly (As a [type of user], I want [some goal] so that [some reason])?
- Are non-functional requirements (security, scale, performance) addressed?

4.2 Qualitative Rubric

The application exposes an "Evaluation" tab in the UI featuring a grading rubric across four dimensions:

1. **Completeness (1-5)**: Does the PRD cover all essential aspects of the product?
2. **Clarity (1-5)**: Are the requirements unambiguous and easy for engineering to understand?
3. **Context Grounding (1-5)**: Did the agents accurately utilize the uploaded RAG documents without hallucinating?
4. **Formatting (1-5)**: Did the agents follow the strict Markdown structure requested in the prompts?

During testing with the provided sample data (e.g., the FinTech app with competitor analysis), the system consistently scored high on formatting and context grounding, successfully pulling in specific competitor weaknesses and business rules into the PRD.

---

5. Challenges

Developing Idea2PRD involved overcoming several technical and architectural challenges:

1. **Dependency Conflicts and Versioning**: Managing the ecosystem of LangChain, OpenAI, and Google libraries proved challenging. Specifically, conflicts between `langchain-openai`, `openai`, and `httpx` (e.g., the `proxies` keyword argument error) required strict dependency pinning (`langchain-openai==0.1.20`, `openai==1.35.0`, `httpx==0.27.2`) to ensure compatibility. Python 3.9 syntax limitations (e.g., `X |`

`None` union types) also required refactoring to `Optional[X]`.

2. **RAG Context Window Limits**: Injecting too much context from uploaded PDFs overwhelmed the model, causing it to lose track of the strict formatting instructions (the "lost in the middle" phenomenon). This was solved by tuning the `TOP_K_RETRIEVAL` parameter down to 5 and utilizing the `RecursiveCharacterTextSplitter` to ensure chunks were semantically meaningful.

3. **Agent Bleed-Over**: Initially, the Business Analyst agent would attempt to write User Stories, stepping on the toes of the Product Manager agent. This was mitigated by applying strict negative constraints in the prompt design (e.g., *"Do not write user stories"*).

4. **Groq Provider Limitations**: Integrating Groq as a fast, free LLM provider was successful for inference, but Groq currently lacks an embedding API. The architecture had to be adapted to fall back to Gemini's free embedding model for the RAG pipeline when Groq is selected as the primary chat model.

---

6. Future Enhancements

While Idea2PRD successfully demonstrates a multi-agent RAG workflow, there are several avenues for future improvement:

1. **Cyclic Agent Collaboration (LangGraph)**: Currently, the agents operate in a strict linear sequence (BA -> PM -> Risk). Future versions should implement a cyclic graph (using a framework like LangGraph) where the Risk Reviewer can send the PRD *back* to the PM agent for revision if critical ambiguities are found, creating a self-healing loop.

2. **Advanced RAG Techniques**: The current RAG pipeline uses simple naive similarity search. Implementing advanced techniques like **Multi-Query Retrieval** (expanding the user's prompt into multiple search queries) or **Parent-Document Retrieval**

(embedding small chunks but returning larger parent sections) would improve context grounding.

3. **Interactive Human-in-the-Loop**: Allowing the user to intervene between agent steps. For example, the user could review and approve the Business Analyst's personas before the Product Manager generates user stories based on them.

4. **Export Integrations**: Beyond downloading a Markdown file, the system could integrate with APIs for Jira, Trello, or Notion to automatically create epics and user story tickets directly from the generated PRD.